

Paradigmas y Lenguajes de Programación 2026

Guía de Estudio — Tema 02

Matías Gel

Ciclo lectivo 2026

Índice de Contenidos

1	Guía de Estudio — Tema 02: Sintaxis y Semántica de Lenguajes	1
1.1	1. Introducción al tema	2
1.2	2. Objetivos de aprendizaje	2
1.3	3. Conceptos previos necesarios	3
1.4	4. Desarrollo teórico	3
1.4.1	4.1 ¿Qué es un lenguaje de programación? Sintaxis y semántica	3
1.4.2	4.2 Estructura léxica: lexemas y tokens	4
1.4.3	4.3 Gramáticas formales: BNF	6
1.4.4	4.4 EBNF y diagramas de sintaxis	9
1.4.5	4.5 Semántica: del significado formal al estado del programa	10
1.4.6	4.6 El analizador sintáctico y el pipeline del compilador	12
1.4.7	4.7 Gramáticas en LLMs modernos — constrained decoding	13
1.5	5. Ejemplos trabajados	14
1.5.1	Ejemplo 1 — Derivación y árbol para $B := (A + C)$	14
1.5.2	Ejemplo 2 — Detección de ambigüedad	15
1.5.3	Ejemplo 3 — Semántica operacional con condicional	16
1.6	6. Puntos clave y resumen	17
1.7	7. Autoevaluación	18
1.8	8. Glosario	19
1.9	9. Referencias y lecturas recomendadas	21
1.9.1	Fuentes principales (usadas en esta guía)	21
1.9.2	Fuente sobre LLMs y gramáticas	21
1.9.3	Lectura opcional para profundizar	21

1 Guía de Estudio — Tema 02: Sintaxis y Semántica de Lenguajes

Materia: Paradigmas y Lenguajes de Programación 2026 **Institución:** Universidad Nacional de Tierra del Fuego — Instituto IDEI **Agente:** Dra. Sofía (study-guide-writer) **Fecha de generación:** 2026-03-11 **Estado:** APROBADO **Aprobado por:** Matías Gel **Fecha de aprobación:** 2026-03-11 **Fuentes integradas:** slides cátedra UNTDF 2025; Sebesta Cap. 3 y 4; Gabbrielli & Martini Cap. 4 §4.1; Willard & Louf

(2023) arXiv:2307.09702 **Para:** Alumno — material de estudio autónomo

1.1 1. Introducción al tema

Este tema ocupa un lugar bisagra en la materia. En el Tema 01 viste que TypeScript compila a JavaScript, que corre en el motor V8. Pero ¿cómo sabe el compilador si tu código está bien escrito? ¿Y qué significa “bien escrito”?

La respuesta tiene dos partes que no son lo mismo:

- **La sintaxis** te dice si el código tiene la *forma* correcta.
- **La semántica** te dice si el código tiene el *significado* correcto.

Este tema explora ambas, las herramientas formales para especificarlas (gramáticas BNF y EBNF), el rol del compilador al procesarlas, y un cierre con la aplicación más inesperada: estas mismas gramáticas son la infraestructura con la que los LLMs modernos garantizan outputs estructurados.

Conexión con el mapa de la materia: - ← Tema 01: máquinas abstractas, intérpretes vs. compiladores - → Tema 03: sistemas de tipos (la semántica en acción) - → Tema 09: variables, binding y ámbito (los nombres en detalle)

1.2 2. Objetivos de aprendizaje

Al terminar de estudiar este tema, podrás:

1. **Distinguir** un error sintáctico de un error semántico estático y de uno dinámico, con ejemplos concretos en TypeScript.
 2. **Identificar** lexemas y tokens en una sentencia de código, reconociendo la categoría de cada uno.
 3. **Leer e interpretar** una especificación de gramática en notación BNF y EBNF.
 4. **Derivar** una cadena a partir de un símbolo inicial aplicando producciones, mostrando la tabla de derivación.
 5. **Construir** el árbol sintáctico de una derivación sencilla.
 6. **Detectar** ambigüedad en una gramática y explicar por qué es problemática para el compilador.
 7. **Explicar** el rol del lexer y el parser en el pipeline de compilación.
 8. **Describir** qué es el constrained decoding en LLMs y su relación con las gramáticas EBNF.
-

1.3 3. Conceptos previos necesarios

Antes de leer este tema, revisá que tenés claros:

Concepto	Dónde está
Máquinas abstractas e intérpretes vs. compiladores	Tema 01
Qué hace <code>tsc</code> cuando compilás TypeScript	Tema 01
Qué es un autómata de estados finitos (solo nivel introductorio)	Álgebra/Automatas

Si el concepto de “compilar” todavía te resulta vago, releer el Bloque 3 del Tema 01 antes de continuar.

1.4 4. Desarrollo teórico

1.4.1 4.1 ¿Qué es un lenguaje de programación? Sintaxis y semántica

Referencia: slides cátedra UNTDF 2025; Sebesta Cap. 3 §3.1 (Ver Filminas 1–6)

Definición formal: > “Un lenguaje de programación es una notación formal para describir algoritmos a ser ejecutados por computadoras.” > — *slides cátedra UNTDF 2025*

La palabra clave es **formal**. Formal implica reglas precisas, no convenciones ni intuiciones. Un lenguaje de programación necesita dos capas de reglas:

Definición — Sintaxis Conjunto de reglas que determinan cuándo un programa está *bien formado*. La sintaxis se ocupa de la **forma**, no del comportamiento. Describe cómo deben combinarse los símbolos para producir programas válidos.

Definición — Semántica Le asigna **significado** a los programas sintácticamente correctos. Responde: ¿qué hace este programa cuando se ejecuta?

La distinción es más profunda de lo que parece en un primer momento. Considerá estos tres ejemplos en TypeScript:

```
// Caso A: Error de SINTAXIS
if true { console.log("ok") }
// tsc: "error TS1005: ')' expected"
```

```
// → El parser no puede construir el árbol porque falta el paréntesis obligatorio.

// Caso B: Error SEMÁNTICO ESTÁTICO (en compilación)
const x: number = "hola"
// tsc: "error TS2322: Type 'string' is not assignable to type 'number'"
// → El código tiene forma correcta, pero el type-checker detecta la inconsistencia de tipos.

// Caso C: Error SEMÁNTICO DINÁMICO (en runtime)
const y = undefined
y.length // TypeError: Cannot read properties of undefined
// → tsc no puede detectarlo en compilación: depende del valor en ejecución.
```

Tabla de clasificación de errores:

Tipo	¿Quién detecta?	¿Cuándo?	Ejemplo TypeScript
Sintáctico	Parser (tsc)	Compilación	<code>if true { }</code> (falta <code>()</code>)
Semántico estático	Type-checker (tsc)	Compilación	<code>const x: number = "hola"</code>
Semántico dinámico	Runtime (V8)	Ejecución	<code>undefined.length</code>

Punto critical: El compilador detecta **siempre** los errores sintácticos y **solo algunos** errores semánticos (los estáticos). Los errores semánticos dinámicos requieren ejecutar el programa para aparecer.

Criterios de buena sintaxis [slides cátedra UNTDF 2025]: - **Legibilidad** — el código se puede leer y entender - **Facilidad de escritura** — las construcciones son naturales para el programador - **Facilidad de verificación** — el compilador puede detectar automáticamente si el programa es válido - **Facilidad de traducción** — el compilador puede generar código eficiente - **Carencia de ambigüedad** — cada programa tiene un único significado posible

El criterio de *facilidad de verificación* es el que habilita que **tsc** pueda darte un error claro con número de línea y columna. El criterio de *carencia de ambigüedad* es el que hace que ese error tenga un único significado correcto.

1.4.2 4.2 Estructura léxica: lexemas y tokens

Referencia: Sebesta Cap. 4 §4.2; slides cátedra UNTDF 2025 (Ver Filminas 7–14)

El compilador recibe tu programa como una cadena de caracteres. El primer paso es identificar las “palabras” que la componen. Esto lo hace el **analizador léxico** (lexer o scanner).

1.4.2.1 4.2.1 Reglas léxicas vs. reglas sintácticas El análisis de un programa se divide en dos niveles:

- **Reglas léxicas:** definen el alfabeto del lenguaje y cómo combinar caracteres para formar palabras válidas. Ejemplo: Java distingue mayúsculas y minúsculas (`myVar` `myvar`); Python trata la indentación como elemento léxico significativo.
- **Reglas sintácticas:** definen cómo combinar esas palabras (tokens) en sentencias válidas.

Esta separación no es arbitraria. Sebesta (§4.1) identifica tres razones:

1. **Simplicidad:** el parser opera sobre tokens abstractos, no caracteres crudos — es mucho más simple.
2. **Eficiencia:** el lexer puede usar autómatas muy rápidos para reconocer patrones léxicos.
3. **Portabilidad:** changeiar el conjunto de caracteres (ASCII vs. Unicode) solo afecta al lexer, no al parser.

1.4.2.2 4.2.2 Lexemas y tokens

Definición — Lexema Unidad sintáctica de más bajo nivel: la cadena de caracteres concreta tal como aparece en el código fuente.

Definición — Token Categoría abstracta a la que pertenece un lexema. El token es la *clase* del lexema.

La relación es la misma que entre instancia y clase: "hola" es un lexema concreto que pertenece al token abstracto `cadena_literal`.

Ejemplo de tokenización — la sentencia `result = oldsum - value / 100;` [Sebesta §4.2]:

Lexema	Token
<code>result</code>	IDENT
<code>=</code>	ASSIGN_OP
<code>oldsum</code>	IDENT
<code>-</code>	SUB_OP
<code>value</code>	IDENT
<code>/</code>	DIV_OP
<code>100</code>	INT_LIT
<code>;</code>	SEMICOLON

Notar: **result**, **oldsum** y **value** son **lexemas distintos** pero comparten el mismo **token** (IDENT). El lexer no sabe si **result** es una variable, una función o una constante — esa distinción la resuelve el analizador semántico más adelante en el pipeline.

1.4.2.3 4.2.3 Elementos sintácticos de un lenguaje Los elementos léxicos que puede tener un lenguaje [slides cátedra UNTDF 2025]:

- **Caracteres** — el alfabeto base
- **Identificadores** — nombres definidos por el programador
- **Operadores** — +, -, *, /, ==, etc.
- **Palabras clave y reservadas** — distinción importante:
 - *Palabra reservada*: no puede usarse como identificador en ningún contexto (ej: **if**, **while**, **const** en TypeScript)
 - *Palabra clave*: tiene un significado especial, pero algunos lenguajes permiten reusarla como identificador en otros contextos
- **Comentarios** — ignorados por el parser
- **Espacios en blanco** — generalmente ignorados (excepto en Python donde la indentación es léxicamente significativa)
- **Delimitadores** — {, }, ;, ,, (,)
- **Expresiones y sentencias**

1.4.3 4.3 Gramáticas formales: BNF

Referencia: Sebesta Cap. 3 §3.2–3.3; slides cátedra UNTDF 2025 (Ver Filminas 15–26)

1.4.3.1 4.3.1 ¿Por qué gramáticas formales? Antes de BNF, el lenguaje FORTRAN (1957) se especificaba en inglés natural. El problema: el inglés es ambiguo. Dos compiladores de FORTRAN podían comportarse diferente ante el mismo programa.

En 1960, John Backus y Peter Naur desarrollaron la **Backus-Naur Form** para especificar Algol 60. Desde entonces, todos los lenguajes de programación serios se especifican formalmente con alguna variante de BNF.

Una gramática formalmente especificada permite **verificar automáticamente** si un programa pertenece al lenguaje — es la base del parser del compilador.

1.4.3.2 4.3.2 Gramáticas libres de contexto Una gramática libre de contexto es la tupla (N, T, S, P):

Elemento	Nombre	Descripción
N	No terminales	Abstracciones, categorías, “clases de construcciones”
T	Terminales	Los lexemas/tokens reales que aparecen en el código
S	Símbolo inicial	El punto de partida de toda derivación
P	Producciones	Las reglas que dicen cómo expandir no terminales

1.4.3.3 4.3.3 Notación BNF Metasímbolos de BNF:

Símbolo	Significado
<code>::=</code>	“se define como”
<code>\ </code>	alternativa (“o bien”)
<code>< ></code>	encierra un símbolo no terminal

Regla: el lado izquierdo (LHS) tiene exactamente un no terminal. El lado derecho (RHS) es la definición de ese no terminal.

Gramática de ejemplo (usada en clase):

```

<assign> ::= <id> := <expr>
<id>      ::= A | B | C
<expr>    ::= <id> + <expr> | <id> * <expr> | (<expr>) | <id>

```

Los terminales son: `:=`, `+`, `*`, `(`, `)`, `A`, `B`, `C`. Los no terminales son: `<assign>`, `<id>`, `<expr>`. El símbolo inicial es `<assign>`.

1.4.3.4 4.3.4 Derivaciones

Definición — Derivación El proceso de reemplazar sucesivamente no terminales usando las producciones, hasta obtener una cadena de solo terminales.

Definición — Forma de sentencia La cadena intermedia en cada paso de la derivación (puede contener terminales y no terminales mezclados).

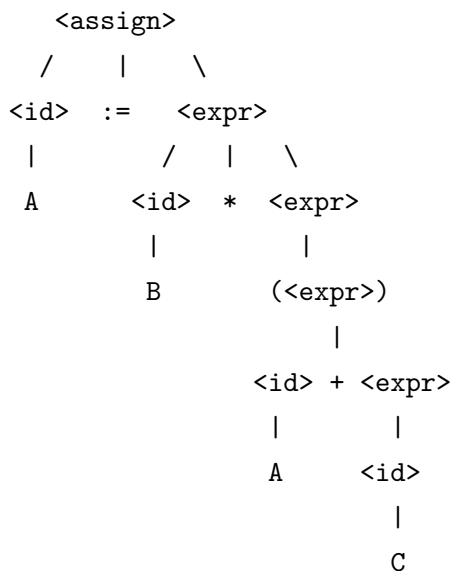
Derivación completa de `A := B * (A + C)`:

Forma de sentencia	Regla aplicada
<code><assign></code>	— (símbolo inicial)

Forma de sentencia	Regla aplicada
<code><id> := <expr></code>	<code>assign → id := expr</code>
<code>A := <expr></code>	<code>id → A</code>
<code>A := <id> * <expr></code>	<code>expr → id * expr</code>
<code>A := B * <expr></code>	<code>id → B</code>
<code>A := B * (<expr>)</code>	<code>expr → (expr)</code>
<code>A := B * (<id> + <expr>)</code>	<code>expr → id + expr</code>
<code>A := B * (A + <expr>)</code>	<code>id → A</code>
<code>A := B * (A + <id>)</code>	<code>expr → id</code>
<code>A := B * (A + C)</code>	<code>id → C</code>

La derivación tiene **10 pasos** (incluyendo el símbolo inicial como punto de partida). Cada paso aplica exactamente una producción.

1.4.3.5 4.3.5 Árboles sintácticos El árbol de derivación (*parse tree*) para `A := B * (A + C)`:



- **Nodos internos** = no terminales
- **Hojas** = terminales (los tokens del código fuente)
- La **jerarquía** del árbol define qué se evalúa primero: los nodos más profundos se procesan antes.

1.4.3.6 4.3.6 Ambigüedad

Definición — Gramática ambigua Una gramática es ambigua si permite construir dos árboles de derivación distintos para la misma cadena de entrada.

Ejemplo: con la gramática $\langle \text{expr} \rangle ::= \langle \text{expr} \rangle + \langle \text{expr} \rangle \mid \langle \text{expr} \rangle * \langle \text{expr} \rangle \mid a \mid b \mid c$, la expresión $a + b * c$ permite dos árboles:

- **Árbol 1:** $(a + b) * c \rightarrow$ resultado depende de los valores
- **Árbol 2:** $a + (b * c) \rightarrow$ resultado diferente

Dos árboles = dos significados posibles. El compilador no sabe cuál elegir $\rightarrow$ **error de diseño del lenguaje**.

Solución: codificar la precedencia y asociatividad en la gramática misma, introduciendo niveles jerárquicos de no terminales (uno por nivel de precedencia).

1.4.4 4.4 EBNF y diagramas de sintaxis

Referencia: slides cátedra UNTDF 2025; Sebesta Cap. 3 §3.3 (Ver Filminas 27–32)

1.4.4.1 4.4.1 EBNF — Extended BNF EBNF extiende BNF con tres operadores que evitan la necesidad de producciones recursivas auxiliares:

Notación	Significado
$[x]$	x es opcional (aparece 0 o 1 vez)
$\{ x \}$	x se repite (aparece 0 o más veces)
$(a \mid b)$	alternativas dentro de un grupo

Equivalencia BNF EBNF:

En BNF, para expresar “una lista de sentencias” necesitás recursión:

$\langle \text{lista} \rangle ::= \langle \text{sentencia} \rangle \mid \langle \text{sentencia} \rangle \langle \text{lista} \rangle$

En EBNF, la misma idea en una línea:

$\langle \text{lista} \rangle ::= \langle \text{sentencia} \rangle \{ \langle \text{sentencia} \rangle \}$

Gramática de un mini-lenguaje imperativo en EBNF:

```

<programa>    ::= { <sentencia> }
<sentencia>   ::= <asignación> | <condicional> | <loop>
<asignación>  ::= <id> "=" <expr> ";"
<condicional> ::= "if" "(" <expr> ")" <sentencia>
               | "if" "(" <expr> ")" <sentencia> "else" <sentencia>
<loop>        ::= "while" "(" <expr> ")" "{" { <sentencia> } "}"

```

[] para el **else** opcional, { } para el cuerpo del **while**.

1.4.4.2 4.4.2 Diagramas de sintaxis (railroad diagrams) Los diagramas de sintaxis son la representación **gráfica** equivalente a EBNF. Se llaman “railroad” porque muestran los caminos posibles como rieles de tren.

Convención visual: - **Rectángulo** → símbolo no terminal (categoría) - **Óvalo / círculo** → símbolo terminal (token concreto)

Para validar una cadena: recorrés el diagrama de izquierda a derecha. En cada bifurcación elegís un camino. Si llegás al final, la cadena es válida. La documentación oficial de TypeScript, Python y muchos otros lenguajes usa railroad diagrams como especificación de sintaxis.

1.4.5 4.5 Semántica: del significado formal al estado del programa

Referencia: Gabbrielli & Martini Cap. 4 §4.1; slides cátedra (Ver Filminas 33–36)

1.4.5.1 4.5.1 ¿Por qué necesitamos semántica formal? Un programa sintácticamente correcto puede no tener el comportamiento esperado. Sin semántica formal, el comportamiento del lenguaje depende de cada implementación — el mismo programa puede producir resultados distintos en dos compiladores diferentes.

La semántica formal garantiza que la **definición del lenguaje** es la misma independientemente del compilador.

1.4.5.2 4.5.2 Nombres y objetos denotables

Definición — Nombre “Una secuencia de caracteres usada para representar, o denotar, otro objeto.” — *Gabbrielli & Martini §4.1*

Distinción fundamental: **un nombre y el objeto que denota no son la misma cosa**. **fie** es solo una cadena de caracteres — el objeto que denota puede ser una variable, una función, un tipo, etc.

Consecuencias prácticas: - El mismo objeto puede tener más de un nombre → **aliasing** - El mismo nombre puede referir a objetos distintos en distintos contextos → **scope** (Tema 09)

Objetos denotables en un lenguaje de programación [Gabbrielli & Martini §4.1]:

Definidos por el usuario: - Variables, parámetros formales - Procedimientos / funciones - Tipos definidos por el usuario - Etiquetas, módulos, constantes, excepciones

Definidos por el lenguaje: - Tipos primitivos (**number**, **string**, **boolean**) - Operaciones predefinidas (+, -, ===) - Constantes predefinidas (**undefined**, **null**)

Definición — Entorno (environment) El componente de la máquina abstracta que mantiene las asociaciones **nombre** → **objeto** en cada punto de ejecución.

Definición — Binding (ligadura) La asociación entre un nombre y el objeto que denota. `const PI = 3.14159` liga el nombre `PI` al valor `3.14159`.

El mecanismo completo de ligadura — cuándo ocurre, binding estático vs. dinámico, reglas de scope — se estudia en Tema 09.

1.4.5.3 4.5.3 Semántica operacional

Definición — Semántica operacional Define el significado de un programa como la **secuencia de estados** que produce al ejecutarse. Un estado es un conjunto de pares (**nombre**, **valor**) en un instante dado.

Ejemplo 1 — secuencia de asignaciones:

Programa:

```
x := 5
y := x + 1
```

Estado inicial: { }

```
→ x := 5          → estado: { x = 5 }
→ y := x + 1      → estado: { x = 5, y = 6 }
```

Estado final: { x = 5, y = 6 }

Ejemplo 2 — condicional que no ejecuta su rama:

Programa:

```
x := 1
if x > 3 then y := 0
```

Estado inicial: { }

```
→ x := 1          → estado: { x = 1 }
→ evaluar x > 3    → 1 > 3 es FALSO
→ rama no ejecutada → estado: { x = 1 } (sin cambio)
```

Estado final: { x = 1 } ← y nunca fue ligada

La semántica operacional modela no solo lo que cambia, sino también lo que el programa **decide no hacer**. El estado puede permanecer idéntico — y eso también es parte del significado del programa.

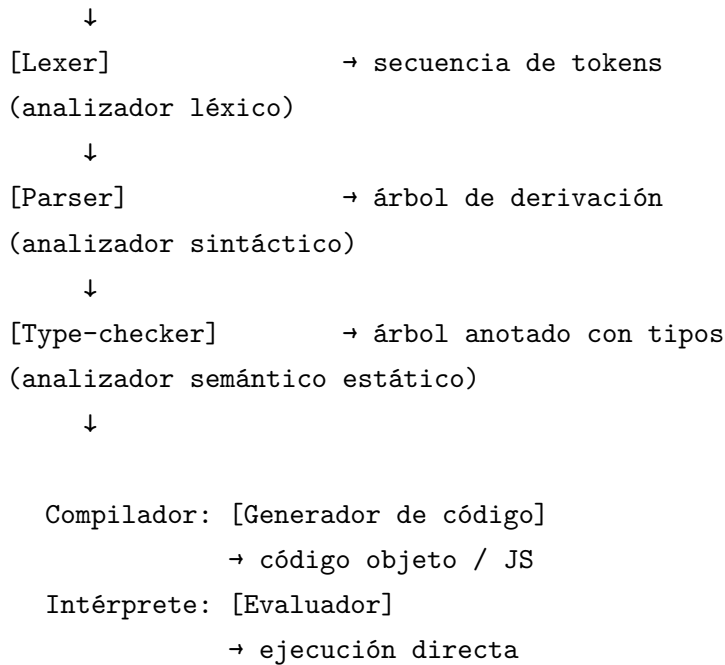
La conexión con el Tema 01: la semántica operacional es la descripción formal del comportamiento de la *máquina abstracta* del lenguaje — exactamente lo que estudiamos como modelo de ejecución.

1.4.6 4.6 El analizador sintáctico y el pipeline del compilador

Referencia: Sebesta Cap. 4 §4.1; slides cátedra (Ver Filminas 37–38)

1.4.6.1 4.6.1 El pipeline completo

Código fuente (texto)



tsc recorre exactamente este pipeline: lexer → parser → type-checker → generador de código JavaScript.

1.4.6.2 4.6.2 Dos estrategias de parsing (conceptual)

Estrategia	Descripción
Top-down (descendente)	Empieza en el símbolo inicial y trata de derivar la cadena de entrada. Intuitivo y legible.
Bottom-up (ascendente)	Parte de los tokens y construye el árbol hacia arriba. Más poderoso, más complejo.

Los algoritmos concretos (recursive-descent, LR) corresponden a Teoría de Compiladores — están fuera del scope de esta materia. Aquí solo necesitamos el concepto.

1.4.7 4.7 Gramáticas en LLMs modernos — constrained decoding

Referencia: Willard & Louf, arXiv:2307.09702 (2023) (Ver Filmina 38)

Esta sección conecta el formalismo que acabás de estudiar con la IA generativa actual.

1.4.7.1 4.7.1 El problema: LLMs y formato libre Un LLM como ChatGPT genera texto token a token eligiendo el siguiente token según probabilidades. Si le pedís que devuelva JSON, el modelo lo *intenta* — pero la instrucción en texto es **semántica**: el modelo la interpreta, y si el texto de entrada es ambiguo o el modelo “cambia de idea” a mitad de la generación, puede producir salidas fuera del formato.

```
// Con el prompt "respondé con JSON" - tres runs, tres formatos posibles:
"El nombre es Juan y tiene 30 años."           ← texto narrativo
{ nombre: "Juan", edad: 30 }                   ← JSON inválido (claves sin comillas)
{"nombre":"Juan","edad":"30"}                  ← edad como string, no número
```

1.4.7.2 4.7.2 La solución: constrained decoding

Definición — Constrained decoding Técnica que compila una gramática EBNF a un autómata de estados finitos y lo usa para filtrar, **token a token durante la generación**, cuáles puede producir el modelo. Solo pueden generarse tokens que pertenezcan a una derivación válida según la gramática.

La diferencia clave:

Enfoque	Mecanismo	Garantía
Instrucción en prompt	Semántico — el modelo la interpreta	Ninguna: el modelo puede desviarse
Constrained decoding	El autómata bloquea tokens inválidos	Estructural: imposible salir de la gramática

El autómata actúa **antes** de que el modelo genere cada token — no valida la respuesta al final. El modelo literalmente no ve los tokens inválidos como opciones.

1.4.7.3 4.7.3 De la teoría a la herramienta

- **Outlines** (Python, open-source): recibe una gramática EBNF como input y aplica constrained decoding sobre cualquier modelo compatible.
- **LMQL**: lenguaje de consulta con restricciones sintácticas declarativas sobre LLMs [Beurer-Kellner et al., VLDB 2023].
- **OpenAI Structured Outputs** (2024): el parámetro `response_format=Persona` en la API implementa exactamente este mecanismo — el esquema Pydantic se compila a JSON Schema, que se compila al autómata.

```
# OpenAI Structured Outputs - el esquema ES la gramática
from pydantic import BaseModel

class Persona(BaseModel):
    nombre: str
    edad: int          # tipo int en la gramática + nunca devolverá "30" como string

response = client.beta.chat.completions.parse(
    model="gpt-4o",
    messages=[{"role": "user", "content": "..."}],
    response_format=Persona,  # + autómata compilado aquí
)

# Resultado: siempre Persona(nombre='Juan', edad=30)
```

La analogía directa: el constrained decoding hace con el LLM lo que el parser hace con el compilador — en cada paso, consulta la gramática para saber qué es válido a continuación y descarta el resto.

1.5 5. Ejemplos trabajados

1.5.1 Ejemplo 1 — Derivación y árbol para $B := (A + C)$

Gramática:

```
<assign> ::= <id> := <expr>
<id>      ::= A | B | C
<expr>    ::= <id> + <expr> | <id> * <expr> | (<expr>) | <id>
```

Tabla de derivación:

Paso	Forma de sentencia	Regla
0	<assign>	— (símbolo inicial)
1	<id> := <expr>	assign $\rightarrow$ id := expr
2	B := <expr>	id $\rightarrow$ B
3	B := (<expr>)	expr $\rightarrow$ (expr)
4	B := (<id> + <expr>)	expr $\rightarrow$ id + expr
5	B := (A + <expr>)	id $\rightarrow$ A
6	B := (A + <id>)	expr $\rightarrow$ id
7	B := (A + C)	id $\rightarrow$ C

7 pasos — solo terminales en la última fila

Árbol sintáctico:

```

      <assign>
    /   |   \
  <id>  :=  <expr>
   |           |
   B         (<expr>)
             |
           <id> + <expr>
             |   |
             A   <id>
                 |
                 C

```

1.5.2 Ejemplo 2 — Detección de ambigüedad

Gramática:

<expr> ::= <expr> + <expr> | <expr> * <expr> | a | b | c

Cadena: a + b * c

Árbol 1 (agrupa + primero):

```

      <expr>
    /   |   \
  <expr> * <expr>
  /\

```

```

<expr>+<expr>  c
  |           |
  a           b

```

→ Calcula $(a + b) * c$

Árbol 2 (agrupa $*$ primero):

```

      <expr>
    /  |  \
<expr> + <expr>
 |      /\
a  <expr>*<expr>
   |      |
   b      c

```

→ Calcula $a + (b * c)$

Dos árboles válidos → dos resultados diferentes → gramática ambigua → **el compilador no puede elegir**.

1.5.3 Ejemplo 3 — Semántica operacional con condicional

Programa:

```

x := 3
y := 10
if x < 5 then z := x + y

```

Traza:

Paso	Operación	Estado resultante
0	(estado inicial)	{ }
1	$x := 3$	{ $x = 3$ }
2	$y := 10$	{ $x = 3$, $y = 10$ }
3	Evaluar $x < 5 \rightarrow 3 < 5 \rightarrow$ verdadero	(evaluación, sin cambio de estado)
4	$z := x + y \rightarrow z := 3 + 10$ $= 13$	{ $x = 3$, $y = 10$, $z = 13$ }

Estado final: { $x = 3$, $y = 10$, $z = 13$ }

¿Qué pasaría si $x := 7$? En el Paso 3, $7 < 5$ sería **falso** $\rightarrow$ la rama del **if** no ejecuta $\rightarrow$ el estado final sería $\{ x = 7, y = 10 \}$ (sin z).

1.6 6. Puntos clave y resumen

Mapa conceptual del tema:

LENGUAJE DE PROGRAMACIÓN

SINTAXIS forma del programa

Léxica: caracteres $\rightarrow$ lexemas $\rightarrow$ tokens (lexer)

Formal: tokens $\rightarrow$ árbol derivación (parser)

Especificada en: BNF / EBNF / railroad diagrams

Propiedad clave: carencia de ambigüedad

SEMÁNTICA significado del programa

Estática: verificación de tipos en compilación (type-checker)

Dinámica: comportamiento en ejecución (runtime)

Operacional: secuencia de estados (máquina abstracta)

Lista de puntos clave:

- La **sintaxis** define forma; la **semántica** define significado — son independientes
 - Un programa puede ser **sintácticamente correcto y semánticamente incorrecto**
 - El compilador detecta **todos** los errores sintácticos pero **solo** los semánticos estáticos
 - Un **lexema** es la cadena concreta; un **token** es su categoría abstracta
 - El **lexer** no sabe si un identificador es variable, función o constante — eso es semántica
 - Una gramática BNF tiene: no terminales $< >$, terminales, símbolo inicial, producciones $::=$
 - Una **derivación** aplica una producción por paso; la **forma de sentencia** es el estado intermedio
 - Una gramática **ambigua** produce dos árboles para la misma cadena — esto es un defecto
 - EBNF agrega $[]$ (opcional) y $\{ \}$ (repetición) para evitar recursión auxiliar
 - La semántica operacional modela el programa como **transformación de estados**
 - El **constrained decoding** aplica gramáticas EBNF en tiempo real durante la generación de LLMs
-

1.7 7. Autoevaluación

Estos ejercicios son distintos al TP. No hay que entregarlos — son para verificar tu comprensión antes del examen.

1. Clasificá cada error con el término exacto (sintáctico / semántico estático / semántico dinámico):

```
// a)
function foo(
  return 42
}

// b)
const arr: string[] = []
arr.push(42)

// c)
const items = ["a", "b"]
console.log(items[10].toUpperCase())

// d)
let x: boolean = true
x = "no"
```

2. Dada la tokenización de `result = oldsum - value / 100;`, ¿cuántas veces aparece el token IDENT? ¿Cuáles son los lexemas correspondientes?

3. Escribí en BNF una gramática para números enteros positivos (secuencias de uno o más dígitos). Usá <dígito> como no terminal para los dígitos del 0 al 9.

4. Usando la gramática de la clase (`<id> ::= A|B|C`, `<expr> ::= <id>+<expr>|<id>*<expr>|(<expr>)|<id>`), ¿pertenece la cadena (A) al lenguaje de `<expr>`? Justificá con una derivación de 3 pasos.

5. Explicá con tus palabras por qué la gramática `<expr> ::= <expr> + <expr> | <id>` es ambigua. ¿Qué consecuencia tiene esto para el compilador?

6. Trazá la semántica operacional del siguiente programa. Mostrá el estado después de cada línea:

```
a := 4
b := a * 2
if b > 6 then c := b - a
```

7. ¿En qué se parecen el parser de un compilador y el autómata de constrained decoding de un LLM? Respondé en 2-3 oraciones usando los términos: gramática, token, derivación válida.

1.8 8. Glosario

Término	Definición
Aliasing	Situación en que un mismo objeto tiene más de un nombre
Ambigüedad	Propiedad de una gramática que permite dos árboles de derivación distintos para la misma cadena
Analizador léxico (lexer)	Componente del compilador que transforma la cadena de caracteres del código fuente en una secuencia de tokens
Analizador sintáctico (parser)	Componente del compilador que recibe tokens y verifica que forman un programa válido según la gramática, construyendo el árbol de derivación
Binding / ligadura	Asociación entre un nombre y el objeto que denota
BNF (Backus-Naur Form)	Notación formal para especificar gramáticas libres de contexto; usa <code>::=</code> , <code> </code> y <code>< ></code>
Constrained decoding	Técnica que compila una gramática EBNF a un autómata para filtrar tokens válidos durante la generación de un LLM
Derivación	Secuencia de pasos que parte del símbolo inicial y aplica producciones hasta obtener una cadena de terminales
EBNF (Extended BNF)	Extensión de BNF con <code>[]</code> (opcional) y <code>{ }</code> (repetición)
Entorno (environment)	Parte de la máquina abstracta que mantiene las asociaciones nombre $\rightarrow$ objeto en cada punto de ejecución
Error semántico dinámico	Error de significado detectable solo en tiempo de ejecución (ej: <code>undefined.length</code>)
Error semántico estático	Error de significado detectable en compilación sin ejecutar el programa (ej: asignar <code>string</code> a <code>number</code>)
Error sintáctico	Error de forma: el programa no respeta las reglas de la gramática (ej: paréntesis faltante)
Forma de sentencia	Cadena intermedia en una derivación, que puede contener terminales y no terminales mezclados

Término	Definición
Gramática libre de contexto	Gramática de la Clase 2 de Chomsky, especificada por la tupla (N, T, S, P)
Lexema	Unidad sintáctica de más bajo nivel: la cadena de caracteres concreta en el código fuente
No terminal	Símbolo abstracto de una gramática que representa una categoría y se define mediante producciones
Objeto denotable	Objeto al que se le puede dar un nombre en un lenguaje de programación (variable, función, tipo, etc.)
Palabra clave	Símbolo con significado especial en el lenguaje (algunos lenguajes permiten su reuso como identificador)
Palabra reservada	Símbolo con significado especial que no puede usarse como identificador en ningún contexto
Pipeline de compilación	Secuencia: código fuente → lexer → parser → type-checker → generador de código
Producción	Regla de una gramática que define cómo expandir un no terminal
Semántica	Capa de un lenguaje que asigna significado a los programas sintácticamente correctos
Semántica operacional	Enfoque que define el significado de un programa como la secuencia de estados que produce
Símbolo inicial	El no terminal de partida de toda derivación en una gramática
Sintaxis	Capa de un lenguaje que define cuándo un programa está bien formado
Terminal	Símbolo de una gramática que representa un token real del lenguaje fuente (no se expande)
Token	Categoría abstracta a la que pertenece un lexema; la “clase” del lexema
Árbol sintáctico (parse tree)	Representación jerárquica de una derivación; nodos internos = no terminales, hojas = terminales

1.9 9. Referencias y lecturas recomendadas

1.9.1 Fuentes principales (usadas en esta guía)

- **Sebesta, R.** (2019). *Concepts of Programming Languages* (12^a ed.). Pearson.
 - Cap. 3: Describing Syntax and Semantics — BNF, EBNF, gramáticas, árboles sintácticos, ambigüedad
 - Cap. 4: Lexical and Syntax Analysis — lexer, parser, pipeline de compilación
- **Gabbrielli, M. & Martini, S.** (2023). *Programming Languages: Principles and Paradigms*. Springer.
 - Cap. 4, §4.1: Names and Denotable Objects — nombres, entorno, binding, aliasing
- **Slides cátedra UNTDF** (2025). *Sintaxis de Lenguajes de Programación*.

1.9.2 Fuente sobre LLMs y gramáticas

- **Willard, B. T. & Louf, R.** (2023). *Efficient Guided Generation for Large Language Models*. arXiv:2307.09702.
- **Beurer-Kellner, L. et al.** (2023). *Prompting Is Programming: A Query Language (LMQL)*. VLDB 2023.
- **OpenAI** (2024). *Structured Outputs*. <https://platform.openai.com/docs/guides/structured-outputs>

1.9.3 Lectura opcional para profundizar

- **Chomsky, N.** (1956). *Three models for the description of language*. IRE Transactions on Information Theory. — el artículo fundacional de la jerarquía de gramáticas.
- **Aho, A., Lam, M., Sethi, R. & Ullman, J.** (2006). *Compilers: Principles, Techniques, and Tools* (2^a ed.) [“Dragon Book”]. — la referencia estándar para quien quiera profundizar en compiladores.